Paper Review:

Exploring Complex Pattern Formation with Convolutional Neural Network

Am. J. Phys. 90, 141–151 (2022)

Rajat Shuvra Saha
Roll No: 2211083
Guide: Dr Colin Benjamin

As Part of P452 Computational Project

Why Pattern Formation?

Pattern formation in Nonequilibrium Systems:

Many physical, chemical, and biological systems operating **far from thermodynamic equilibrium** spontaneously generate structured spatiotemporal patterns.

Examples:

1. Chemical reaction systems

Autocatalytic reactions and oscillatory kinetics (e.g., Turing patterns, chemical waves)

2. **Biological morphogenesis**

Spatial organization of tissues and pigmentation patterns during development

3. **Ecological systems**

Vegetation banding and population clustering in arid landscapes

Conus textile **exhibits a cellular automaton pattern on its shell**

Natural Turing Pattern on a giant pufferfish

Reaction-Diffusion Equations:

A broad class of nonequilibrium systems can be modeled by nonlinear reaction–diffusion equations:

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u + R(u,v)$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + S(u,v)$$

u, v : interacting species concentrations
D_u, D_v : diffusion coefficients
$R(u,v),\ S(u,v)$: nonlinear reaction kinetics

$$\text{Diffusion} + \text{Nonlinear Reaction} \implies \text{Instability} \implies \text{Pattern Formation}$$

The Gray Scott Model:

The Gray–Scott Model

The Gray–Scott model consists of two coupled nonlinear reaction–diffusion equations:

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u - uv^2 + f(1-u)$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + uv^2 - (f+k)v$$

u, v : concentrations, D_u, D_v : diffusion coefficients, $f, k > 0$: reaction rates

Homogeneous steady states $(\partial_t u = \partial_t v = 0,\ \nabla^2 u = \nabla^2 v = 0)$

$$(u,v) = (1,0)$$

$$(u,v) = \left(\frac{\pm\sqrt{C} + f}{2f},\ \frac{\mp\sqrt{C} - f}{2(f+k)} \right)$$

where

$$C = -4fk^2 - 8f^2k - 4f^3 + f^2$$

Nontrivial steady states exist only if:

$$C \geq 0, \quad f \leq \frac{1}{4}, \quad k \leq \frac{\sqrt{f} - 2f}{2}$$

Numerical Implementation of GS Model:

Spatial and Temporal Discretization

We solve the Gray–Scott equations using the **Forward Time Centered Space (FTCS)** scheme.

$$x = j\Delta, \quad y = i\Delta, \quad t = n\,dt, \quad i, j, n \in \mathbb{N}$$

Grid size:

$$128 \times 128, \quad \Delta = 1, \quad dt = 0.25$$

Discrete Update Equation (for u)

$$u_{i,j}^{n+1} = u_{i,j}^{n} + \frac{D_u\,dt}{\Delta^2}\left(u_{i+1,j}^{n} + u_{i-1,j}^{n} + u_{i,j+1}^{n} + u_{i,j-1}^{n} - 4u_{i,j}^{n}\right) + dt\,r(u_{i,j}^{n}, v_{i,j}^{n})$$

where

$$r(u, v) = -uv^2 + f(1 - u)$$

(Analogous equation for v.)

Boundary Conditions

Periodic boundaries:

$$u_{0,j} = u_{N,j}, \quad u_{i,0} = u_{i,N}$$

Indices wrap at domain edges.

Data Preparation for CNN

For each simulation:

$$\{u(t), u(t + \Delta T), u(t + 2\Delta T)\}$$

with

$$\Delta T = 30$$

Stored as:

$$128 \times 128 \times 3$$

Interpreted as RGB image:

Stationary → Gray Dynamic → Color shifts

Classification of Pattern via Neural Networks:

The Challenge:

- ~15 known Gray–Scott pattern classes (literature-based)
- Traditionally classified visually
- No simple order parameter
- Large parameter space (f, k)
- Exhaustive scanning is computationally expensive

Limitations of Classical Feature Methods

- Fourier transforms
- Statistical measures
- Integral-geometric descriptors
- ➞ Work only in special cases
  ➞ Hard to generalize

Reformulate pattern identification as an image classification problem.

Why Not Linear Regression?

Simple Linear Model:

$$y = w^T x$$

Problems

- Input dimension:

$$128^2 \times 3 = 49{,}152$$

- Too many parameters in dense model
- Strong nonlinear spatial correlations
- Output is categorical (classification), not continuous

Linear models cannot capture complex spatial morphology.

Fully Connected Neural Networks:

Layer Transformation

$$x_j^{(n+1)} = h\left(\sum_{i=1}^{D} w_{ji}^{(n+1)}\, x_i^{(n)} + b_j^{(n+1)}\right)$$

$h(z)$:activation function (ReLU)
Hidden layers extract nonlinear features

Output Layer (Softmax):

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

p_k :probability pattern belongs to class k

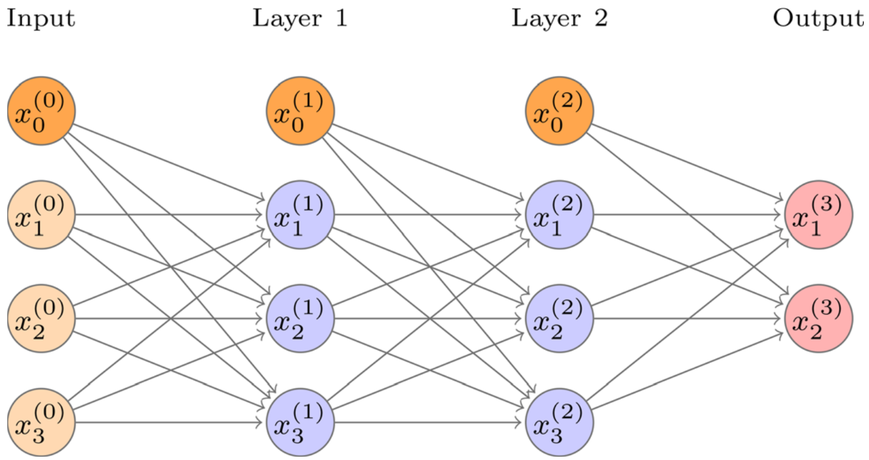


Fig 1: Illustration of a simple fully connected neural network with three input values, two hidden layers with three neurons per layer, and two output classes. Each layer contains a bias unit to allow for nonzero intercepts. Each arrow corresponds to a trainable weight. The hidden layers allow the network to be trained, but, because these layers typically do not represent a simple relation, they are not reported as output.

Loss Function

$$\text{Cross Entropy} = -\sum_{l=1}^{L}\sum_{k=1}^{K} s_{lk} \log\left(p_k(\mathbf{x}_l)\right)$$

Minimized during training.

Why Convolutional Neural Networks?

Curse of Dimensionality

Dense network ignores spatial structure

Images have local correlations

Convolution Operation

$$(\mathbf{K} * \mathbf{X})(i, j) = \sum_{m,n} K_{mn} X_{i+m,j+n}$$

Kernel extracts local features

Parameters shared across image

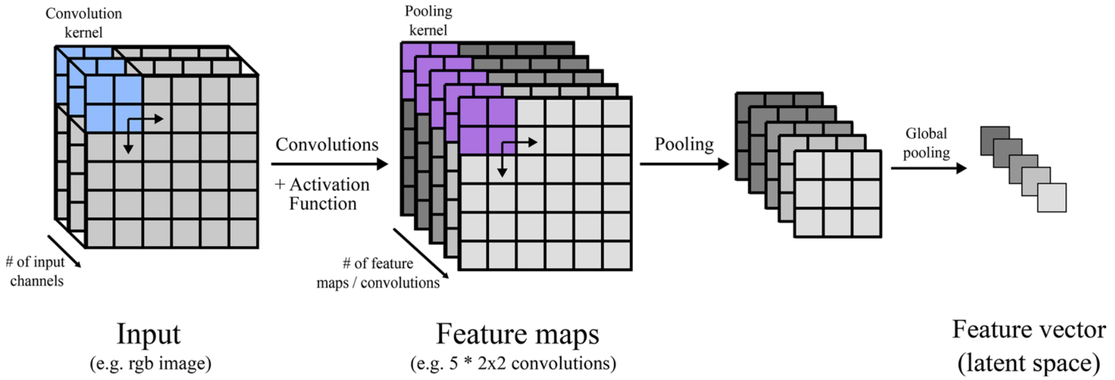


Figure 2:
Illustration of a two-dimensional convolution and pooling operation with a three-channel input image. The convolution kernel size needs to be defined in the horizontal and vertical directions, but the depth always corresponds to the number of channels in the input. Here, we use five convolution kernels of size 2 x2 x3, which results in five feature maps.

Dimensionality Reduction

Max pooling:

$$2 \times 2 \Rightarrow \textit{size reduced by factor 4}$$

Global max pooling $\rightarrow$ feature vector

Key Idea

CNN automatically learns morphological features.

Architecture of the CNN Used:

Layer	Kernel size	Output	Activation	Parameters
Batch normalization	...	128, 128, 3	...	12
Conv2D	3, 3 (2 s)	63, 63, 32	ReLU	896
Conv2D	3, 3	61, 61, 64	ReLU	184 96
MaxPool	2,2	30, 30, 64	...	0
Conv2D	3, 3	28, 28, 128	ReLU	738 56
MaxPool	2, 2	14, 14, 128	...	0
Conv2D	3, 3	12, 12, 128	ReLU	147 584
MaxPool	2, 2	6, 6, 128	...	0
GlobMaxPool	...	128	...	0
Dense	...	256	ReLU	330 24
Dropout	...	256 (85%)	...	0
Dense	...	15	Softmax	3855
...	...	...	...	$\sum = 277\,723$

Figure 3:
Architecture of the two-dimensional convolutional network with layer type, size, and stride s of the convolution kernel, activation function, and the number of parameters (fixed and trainable). The stride s is the amount of pixel shift of the convolution kernel. Conv2D refers to a two-dimensional convolution, MaxPool is a maximum pooling operation, and GlobalMaxPool refers to the global maximum pooling. Dense layers are equivalent to a fully connected neural network. The dropout layer is active only during training and randomly sets 15% of the weights to zero in each iteration. For the last layer, a Softmax activation is applied to convert the output into probabilities.

Iterative Workflow of our Model

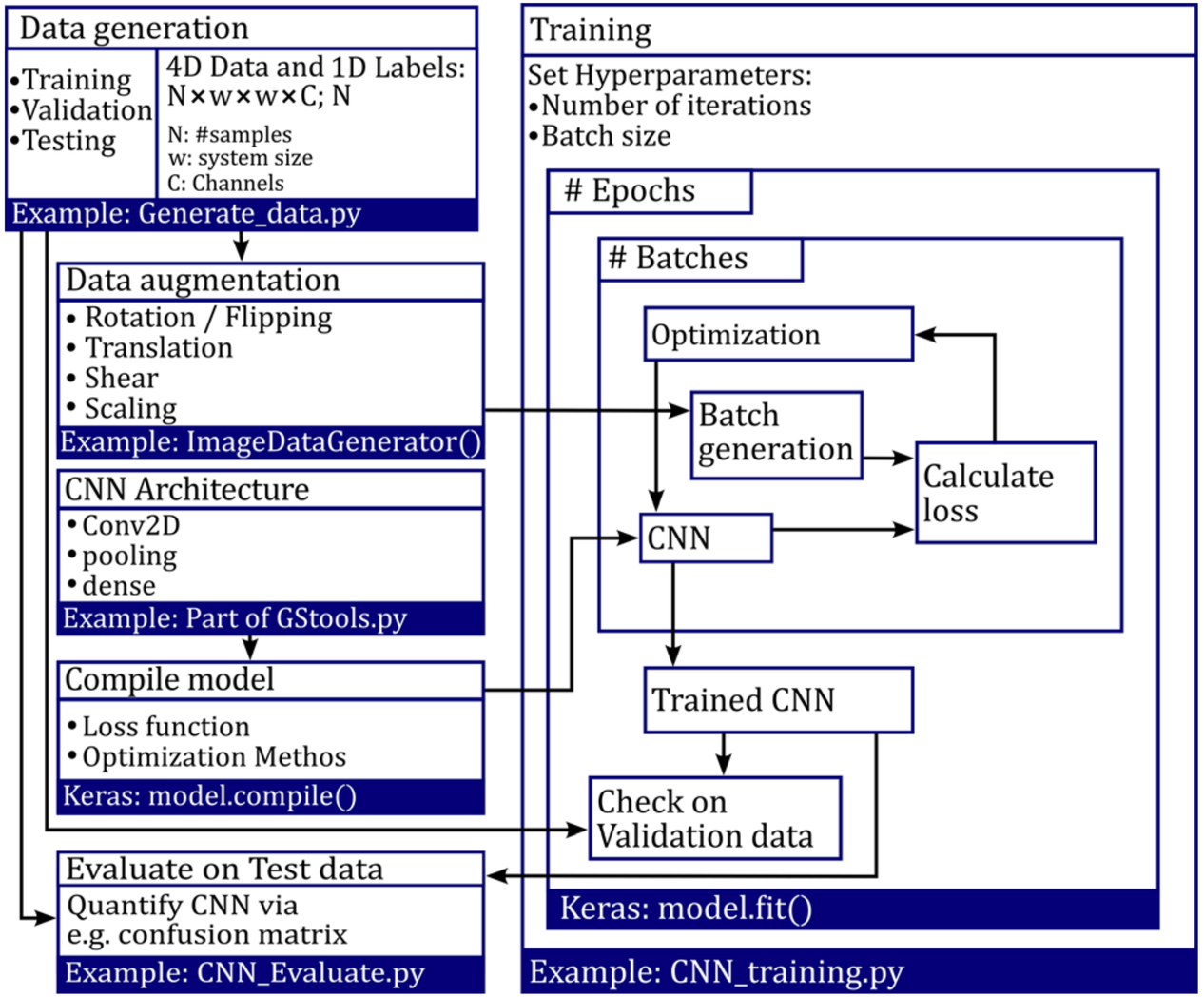


Figure 4:
Illustration of the data generation, training, validation, and test procedure. This workflow is common for high level machine learning. The generated datasets serve as input to the data augmentation function, validation, and test step. The CNN is defined, and the weights of the network are iteratively optimized in the training loop by minimizing the categorical crossentropy. After the training is finished, the predictive capabilities of the CNN are evaluated with the test data.

Data Splitting and Overfitting Control:

Dataset Partitioning
•Data split into:
- **Training set**
- **Validation set**
- **Test set**

Training Set
•Used for optimization of network weights

Validation Set
•Evaluated during training
•Detects overfitting
If:

$$\text{Training } \textit{Accuracy} \gg \textit{Validation Accuracy}$$

→ Model is overfitting

Test Set
•Used only after training
•Evaluates generalization performance

Overfitting Mitigation
•Reduce model complexity
•Dropout regularization
•Data augmentation

Simulation Setup
•128 × 128 grid
•Three stored snapshots:

$$t = 14940,\ 14970,\ 15000$$

System chosen after convergence to steady behavior.

Dataset Size
•**15 classes**
•100 samples per class
•Total: **1500 samples**
•Split evenly:
- 750 training
- 750 validation

Test Dataset
•750 additional patterns
•Slightly varied (f, k)
•Tests parameter-space generalization

Data Format
Each sample stored as:

$$128 \times 128 \times 3$$

Interpreted as RGB image.

Implementation and Computational Cost:

IBM **Quantum**

Single Simulation
Run Gray–Scott model for parameters $[seed]$ $[D_u]$ $[f]$ $[k]$:
`python Gray_Scott_2D.py 2 0.2 0.009 0.045`
•First two arguments optional
•Seed controls random initial conditions

Dataset Generation (2D CNN)
```
python Dataset_Generate.py train 2D
python Dataset_Generate.py val 2D
python Dataset_Generate.py test 2D
```
Runtime (Intel i7-9750H, 2.6 GHz):
•~2 hours per dataset
•Total ≈ 6 hours for train + val + test

Model Training
`python CNN_Train_2D.py`
Training time:
•~1.4 hours
Model saved as:
`model_CNN_2D`

Dense Parameter Scan
`python Parameter_Space_Dataset_Generate.py`
Runtime:
•~24 hours
Classification:
```
python Parameter_Space_Dataset_Classify.py
model_CNN_2D
```

Total Computational Effort
•Initial datasets: ~6 hours
•Training: ~1.4 hours
•Dense scan: ~24 hours
•**Total ≈ 31–32 hours (single CPU)**

Pre-generated Dataset and Model Parameters were ready to download at:

https://osf.io/byrzm/files/yg2q4

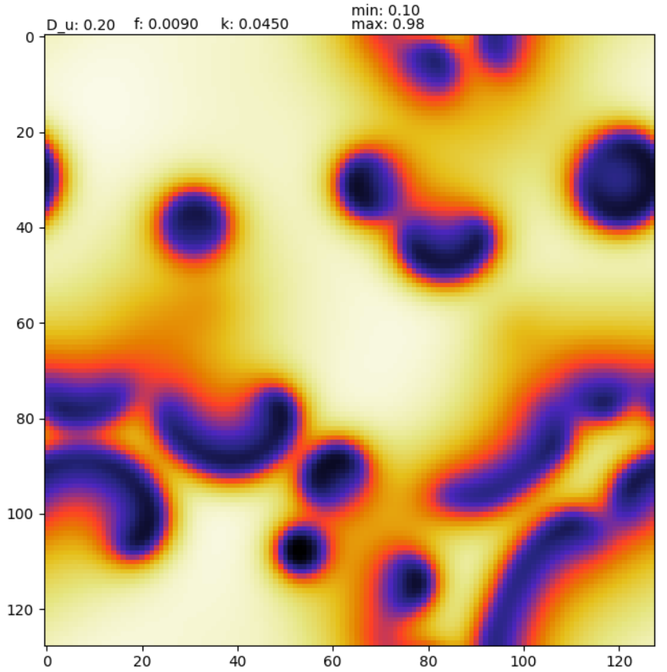


Figure 5: GS Pattern Output for:
D_u=0.20
f: 0.0090
K: 0.0450

Training and Optimization:

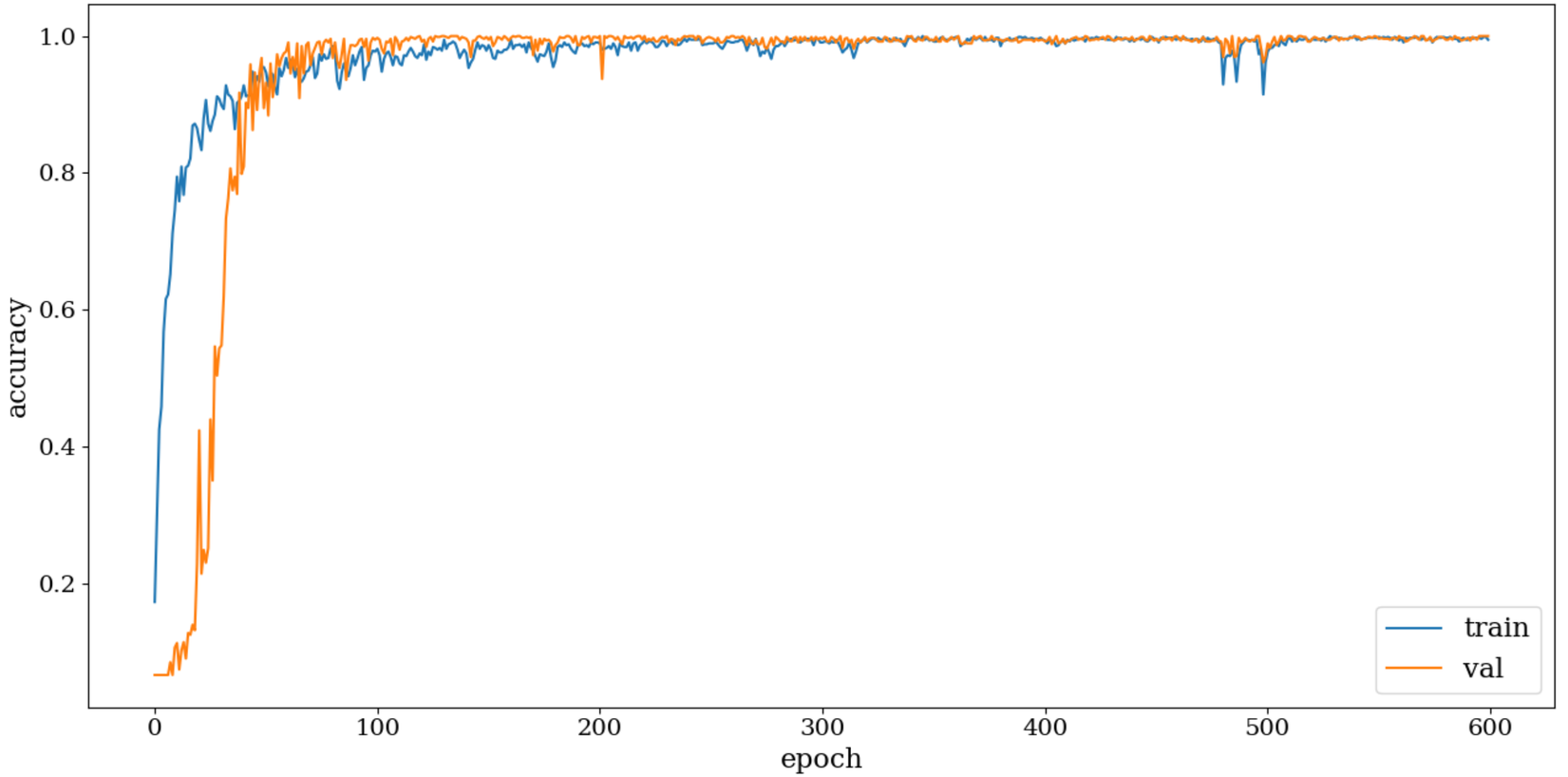


Figure 6:

Training history and accuracy for the training and validation data. The accuracy usually approaches >99%, and we stop the training phase after 600 epochs.

Confusion Matrix:

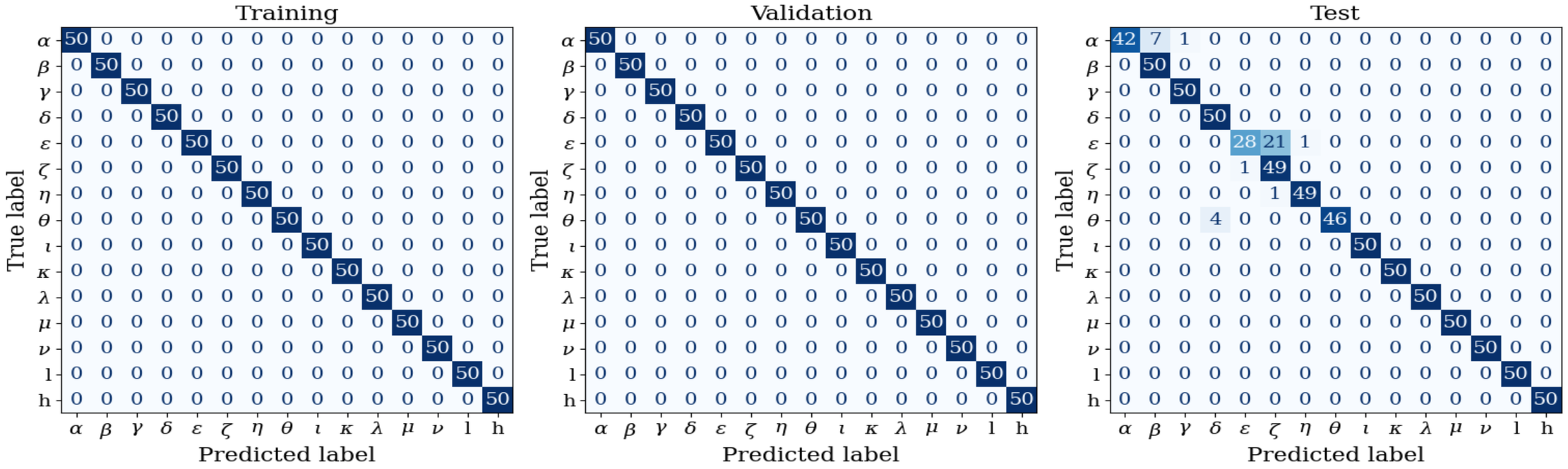


Figure 7:
Confusion matrix for the training, testing and validation dataset; 96% of the patterns are correctly classified

Results: Dense Parameter Space Scan

IBM Quantum

Training vs. Exploration

Training data:

$$\{(f_i, k_i)\}_{i=1}^{15} \quad \text{(per class)}$$

Exploration region:

$$f \in [0.02, 0.08], \qquad k \in [0, 0.12]$$

Total simulations:

$$N = 5481$$

Each sample:

$$\mathbf{X} \in \mathbb{R}^{128\times128\times3}$$

Prediction Procedure

Neural network output:

$$\mathbf{p}(\mathbf{X}) = (p_1, \ldots, p_{15}), \qquad \sum_{k=1}^{15} p_k = 1$$

Class assignment:

$$\hat{c} = \arg\max_k \; p_k$$

Result:

$$\mathcal{P}(f, k) \longrightarrow \hat{c}(f, k)$$

Connected domains emerge in parameter space.

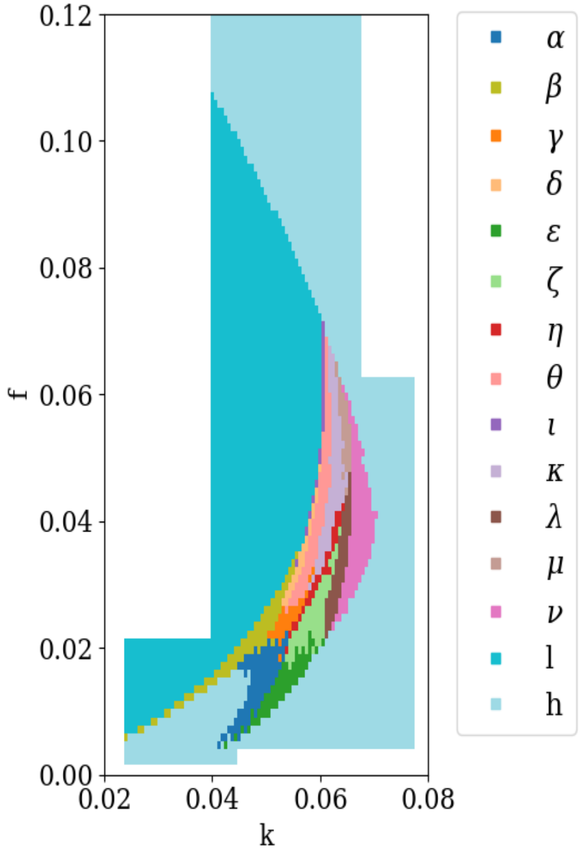


Figure 8: Parameter Scan Plot for First Iteration of Training

Refinement of Training Dataset and Second Pass:

IBM **Quantum**

Results After Refinement

Performance

- Improved test accuracy
- Faster convergence
- Reduced variance across epochs

Phase Diagram Update

$$\hat{c}^{(2)}(f,k)$$

More compact domains.
New class detected at:

$$f \approx 0.068, \qquad k \approx 0.0606$$

Key Insight

CNN does not autonomously define new classes.
However:

$$\text{Misclassification} \Rightarrow \text{Training data refinement}$$

$$\text{Machine Learning} + \text{Physical Interpretation} = \text{Iterative Discovery}$$

Dataset Refinement: Second Training Pass

Observed Issues

- Misclassified patterns
- Class mixtures
- Slowly converging dynamics
- Emergence of new morphology: class p

Refinement Strategy

$$\text{Augment training set} \quad + \quad \text{Introduce class } p$$

New dataset size:

$$N_{\text{train}} = 3200 \quad (200 \text{ per class})$$

Network retrained from scratch.

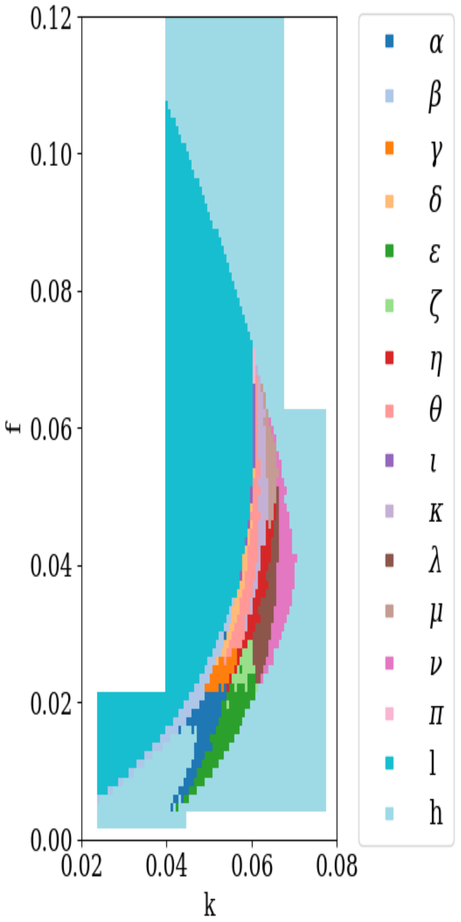
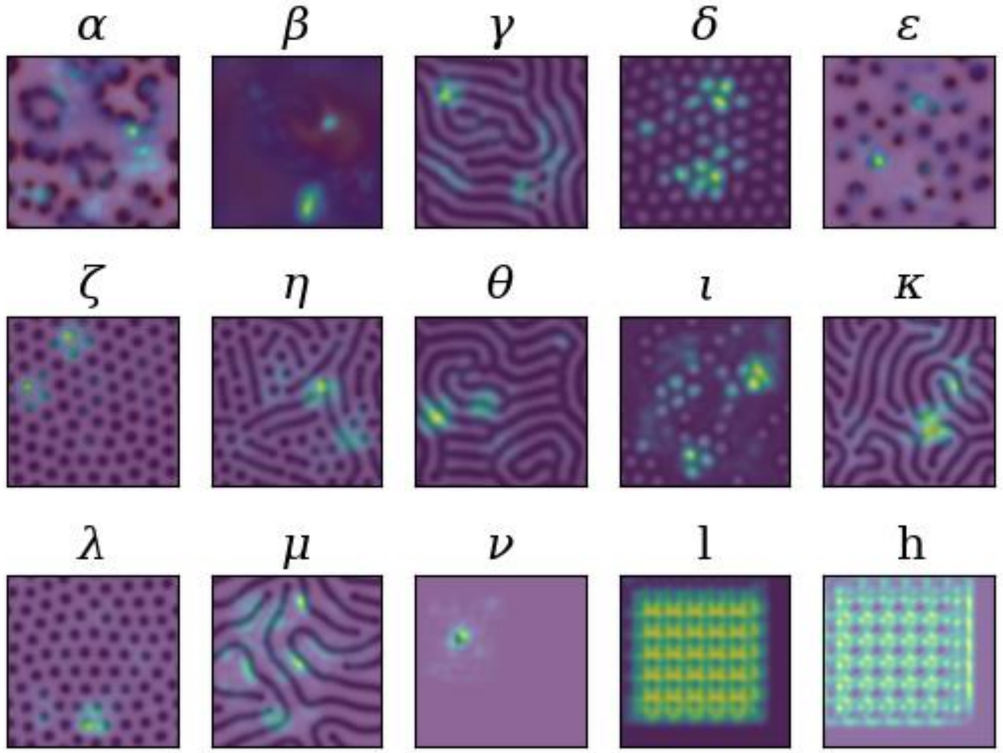
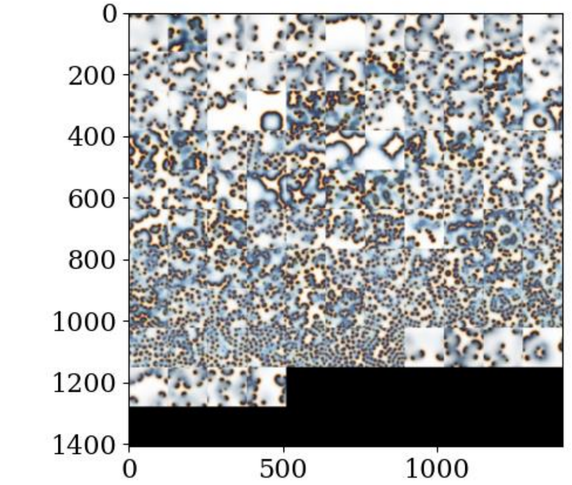


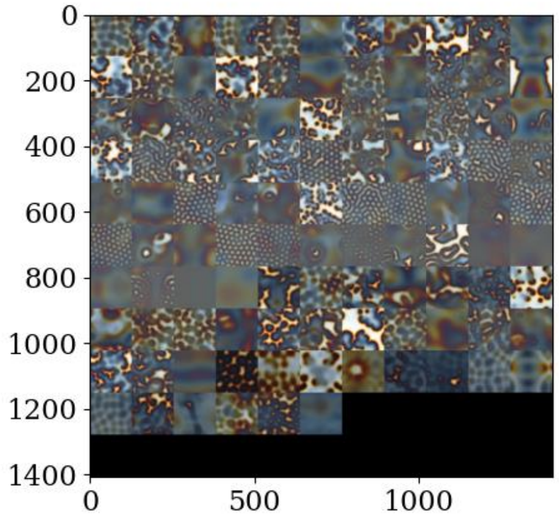
Figure 9:
Parameter Space Scan Plot on Refinement

Montage Visualization for all Classes:



Saliency Plots of all the classes





Montage of Classes 0 and Classes 1 as sample

Discussion of our Results:

Scientific Contribution

- CNN framework applied to Gray–Scott reaction–diffusion system
- Automated reconstruction of phase diagram:

$$(f, k) \mapsto \hat{c}(f, k)$$

- Eliminates manual feature engineering
- Identifies class boundaries in parameter space
- Complements analytical stability analysis

Theoretical Limitations

1. Black-Box Nature

Learned feature representation is not analytically interpretable
No explicit order parameter extracted
Boundaries are empirical, not derived

2. Dependence on Labeled Data

Classification limited to predefined classes
CNN cannot autonomously define new morphologies
Sensitive to dataset bias and sampling density

3. Resolution & Dynamics Constraints

Only 2D spatial convolution used
Time evolution encoded indirectly (3 snapshots)
Fine dynamical distinctions may be under-resolved

Future Directions for Improvements:

1. Improved Temporal Modelling

- 3D convolutional networks:

$$\mathbf{X} \in \mathbb{R}^{128\times128\times T}$$

- Sequence models (Conv-LSTM)
- Explicit spatiotemporal feature extraction

2. Transfer Learning

- Use pretrained CNN backbones
- Retrain final layers on Gray–Scott dataset
- Improve robustness with limited data

3. Physics-Informed Architectures

- Incorporate conservation constraints
- Embed PDE structure into loss function
- Combine CNN with stability theory

4. Continuous Prediction

- Move beyond classification:

$$(f, k) \;\mapsto\; \textit{continuous morphology descriptor}$$

- CNN regression for PDE solution fields
- Direct surrogate modelling of reaction–diffusion dynamics

Bottom Line of our Project:

CNNs do not replace analytical physics. They extend our ability to explore high-dimensional nonlinear systems.

Sometimes it is the people who no one imagines anything of, often do the things no one can imagine.

By,
Alan Mathison Turing,
Father of Modern Computer Science

Thank you for attending this session! Any Questions can be emailed me at rajatshuvra.saha@niser.ac.in